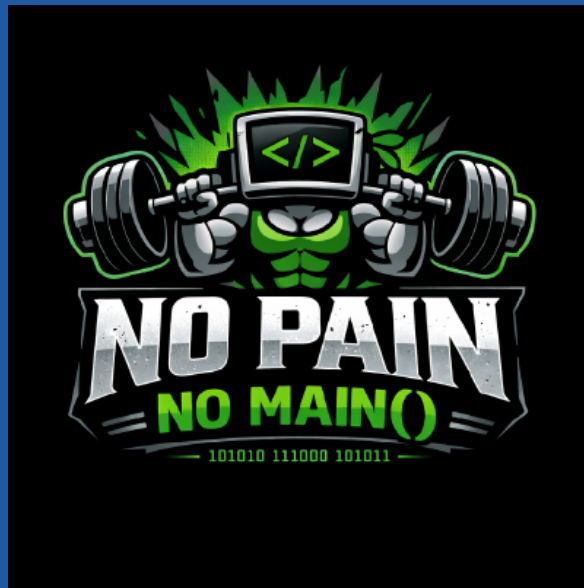


INGENIERÍA DE SOFTWARE 1

Informe de Linter

No Pain No Main()



Autores: Andres Sebastian Sanchez Quintero

Yohan Steven Jiménez Hilarión

Miguel Angel Hincapie Santacruz

Sebastian Fernando Gutiérrez Rojas

Julian Ramirez Diaz

14/06/2026

1. Introducción

Con el objetivo de garantizar la calidad, consistencia y mantenibilidad del código fuente del proyecto **NoPainNoMain**, se realizó un proceso de análisis estático utilizando herramientas de *linting*. Estas herramientas permiten detectar problemas de estilo, convenciones de codificación, errores potenciales y desviaciones respecto a estándares previamente definidos.

El análisis se realizó sobre la totalidad del repositorio empleando la herramienta Checkstyle integrada mediante Maven. Adicionalmente, se utilizó Spotless para aplicar correcciones automáticas relacionadas con formato y organización del código.

2. Herramienta utilizada

La herramienta principal utilizada fue:

Herramienta de análisis estático

Checkstyle mediante el plugin `maven-checkstyle-plugin`.

La ejecución del análisis se realizó utilizando el siguiente comando:

```
.\mvnw.cmd checkstyle
```

Para la corrección automática de problemas de formato se utilizó adicionalmente:

```
.\mvnw.cmd spotless
```

empleando la herramienta Spotless con la configuración Google Java Format.

3. Configuración aplicada

Se utilizó como base la configuración estándar de Google Checkstyle. Sin embargo, fue necesario personalizar la regla encargada de validar los nombres de paquetes debido a la estructura adoptada por el proyecto.

La estructura utilizada en el repositorio sigue el patrón:

```
com.adanext.NoPainNoMain.*
```

Por esta razón se modificó la regla `PackageName` con la siguiente expresión regular:

```
^com.adanext.NoPainNoMain(.[a-z][a-z0-9])$
```

Esta configuración permite mantener el nombre principal del proyecto en formato PascalCase, mientras que los subpaquetes continúan utilizando la convención tradicional en minúsculas.

4. Resultados iniciales

La primera ejecución de Checkstyle permitió identificar una cantidad considerable de advertencias relacionadas principalmente con:

- Formato del código.
- Uso obligatorio de llaves (`NeedBraces`).

- Longitud máxima de línea (LineLength).
- Convenciones de nombres de paquetes (PackageName).
- Documentación JavaDoc (MissingJavadocMethod y MissingJavadocType).

Checkstyle Results			
The following document contains the results of Checkstyle 9.3 with google_checks.xml ruleset.			
Summary			
Files	Info	Warnings	Errors
95	0	2745	0

Figure 1: Resultado inicial del análisis estático.

```

PS C:\Users\PC\Documents\Wo-Pain-No-Main\project-backend\WoPainNoMain> .\mvnw.cmd checkstyle:checkstyle
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.adanext:NoPainNoMain >-----
[INFO] Building 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- checkstyle:3.3.1:checkstyle (default-cli) @ NoPainNoMain ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/skins/maven-default-skin/1.3/maven-default-skin-1.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/skins/maven-default-skin/1.3/maven-default-skin-1.3.jar (14 kB at 31 kB/s)
[INFO] Rendering content with org.apache.maven.skins:maven-default-skin:jar:1.3 skin.
[WARNING] Unable to locate Source XRef to link to - DISABLED
[WARNING] Unable to locate Test Source XRef to link to - DISABLED
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.024 s
[INFO] Finished at: 2026-06-14T19:10:34-05:00
[INFO]
PS C:\Users\PC\Documents\Wo-Pain-No-Main\project-backend\WoPainNoMain> start target\site\checkstyle.html
PS C:\Users\PC\Documents\Wo-Pain-No-Main\project-backend\WoPainNoMain>

```

Figure 2: Evidencia de la ejecución inicial del linter.

5. Correcciones automáticas mediante Spotless

Como primera medida se aplicó la herramienta Spotless con el fin de corregir automáticamente problemas de formato, indentación, espacios en blanco y organización de imports.

```

[INFO] Spotless.Java is keeping 103 files clean - 103 were changed to be clean, 0 were already clean, 0 were skipped because caching determined they were already clean
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 8.464 s
[INFO] Finished at: 2026-06-14T20:16:27-05:00
[INFO]
PS C:\Users\PC\Documents\Wo-Pain-No-Main\project-backend\WoPainNoMain> .\mvnw.cmd spotless:apply

```

Figure 3: Aplicación de correcciones automáticas mediante Spotless.

Checkstyle Results			
The following document contains the results of Checkstyle 9.3 with google_checks.xml ruleset.			
Summary			
Files	Info	Warnings	Errors
94	0	313	0

Figure 4: Resultado posterior a la ejecución de Spotless.

La aplicación de estas correcciones permitió reducir significativamente la cantidad de advertencias detectadas por Checkstyle.

6. Correcciones manuales

Una vez realizadas las correcciones automáticas, se procedió a revisar manualmente las advertencias restantes.

6.1. Regla NeedBraces

Se agregaron llaves explícitas a todas las estructuras de control que no las incluían, incluso en aquellos casos donde Java permite omitirlas.

Las estructuras corregidas incluyen, principalmente:

- if
- else
- continue

Esta modificación mejora la legibilidad y reduce la posibilidad de errores futuros durante tareas de mantenimiento.

6.2. Regla PackageName

Se personalizó la regla de validación de nombres de paquetes para adaptarla a la estructura utilizada por el proyecto, como se mencionó anteriormente.

```
com.adanext.NoPainNoMain.*
```

La modificación permitió eliminar falsos positivos generados por la utilización del nombre del proyecto en formato PascalCase.

6.3. Regla LineLength

Se revisaron y reformatearon todas las líneas que excedían el límite máximo permitido por la configuración de Checkstyle, garantizando el cumplimiento de la longitud máxima establecida. Aunque fuese solo una, es un cambio necesario para mejorar la legibilidad del código.

6.4. Advertencias relacionadas con JavaDoc

Las advertencias correspondientes a:

- MissingJavadocMethod
- MissingJavadocType

fueron evaluadas de manera individual y finalmente descartadas como criterio de corrección obligatoria.

Esta decisión se fundamenta en los principios descritos en *Clean Code*, donde se establece que el código debe ser lo suficientemente expresivo para comunicar su intención sin depender excesivamente de comentarios.

Siguiendo este criterio, el equipo priorizó la claridad de nombres, la correcta separación de responsabilidades y la legibilidad del código antes que la incorporación masiva de

comentarios JavaDoc generados únicamente para satisfacer una regla automática del linter.

Esta decisión fue revisada y aprobada dentro del contexto académico del proyecto, por lo que dichas advertencias no fueron consideradas defectos reales de calidad del software.

Rules			
Category	Rule	Violations	Severity
javadoc	MissingJavadocMethod ⓘ - scope: "public" - tokens: "METHOD_DEF, CTOR_DEF, ANNOTATION_FIELD_DEF, COMPACT_CTOR_DEF" - minLineCount: "2" - allowedAnnotations: "Override, Test"	71	Warning
	MissingJavadocType ⓘ - scope: "protected" - excludeScope: "nothing" - tokens: "CLASS_DEF, INTERFACE_DEF, ENUM_DEF, RECORD_DEF, ANNOTATION_DEF"	94	Warning
	SummaryJavadoc ⓘ forbiddenSummaryFragments: "@return the * \"This method returns \"A {}@code [a-zA-Z0-9]+{} (is a)"	10	Warning
naming	PackageName ⓘ format: "\"com\\.adanext\\.NoPainNoMain \\. [a-z] [a-z0-9]*\""	3	Warning

Figure 5: Resultado después de las correcciones manuales.

Esta ultima verificación permitió encontrar que 3 de los paquetes estaban nombrados de forma inconsistente, por lo que el Linter cumplió su trabajo y se modificaron esos paquetes.

7. Resultado final

Por ultimo, después de aplicar las correcciones automáticas y manuales, se ejecutó nuevamente el análisis estático obteniendo el siguiente resultado:

Rules			
Category	Rule	Violations	Severity
javadoc	MissingJavadocMethod ⓘ - scope: "public" - tokens: "METHOD_DEF, CTOR_DEF, ANNOTATION_FIELD_DEF, COMPACT_CTOR_DEF" - minLineCount: "2" - allowedAnnotations: "Override, Test"	71	Warning
	MissingJavadocType ⓘ - scope: "protected" - excludeScope: "nothing" - tokens: "CLASS_DEF, INTERFACE_DEF, ENUM_DEF, RECORD_DEF, ANNOTATION_DEF"	94	Warning
	SummaryJavadoc ⓘ forbiddenSummaryFragments: "@return the * \"This method returns \"A {}@code [a-zA-Z0-9]+{} (is a)"	10	Warning

Figure 6: Resultado final del análisis estático.

Las advertencias restantes corresponden únicamente a reglas relacionadas con documentación JavaDoc, por lo tanto no es necesario revisar nada más.

8. Conclusiones

La utilización de Checkstyle permitió detectar de forma temprana problemas de formato, consistencia y estilo dentro del proyecto NoPainNoMain.

La combinación de correcciones automáticas mediante Spotless y ajustes manuales permitió eliminar las advertencias que representaban problemas reales de mantenibilidad y legibilidad del código, tales como estructuras sin llaves, nombres de paquetes

incompatibles con la convención del proyecto y líneas que excedían la longitud máxima permitida.

Respecto a las advertencias relacionadas con JavaDoc, se decidió conscientemente no considerarlas como defectos de calidad debido a que, siguiendo los principios de Clean Code, se priorizó la construcción de código autoexplicativo y expresivo sobre la generación masiva de comentarios.

Como resultado, el proyecto alcanzó un nivel de calidad consistente con los estándares definidos para el equipo de desarrollo, manteniendo además una base de código más limpia, legible y fácil de mantener.